

Exercise No. 4

HASH FUNCTIONS

Adam Cherek, s207250

Task 3.1: Hash Function Avalanche Effect

- **MD5:**
 - Original: 7C 8A 80 FB 86 16 C9 5C AD E8 OB CF BF 91 8E 40
 - 1 letter changed: 94 63 16 6A B6 9A ED FA CE 50 88 74 DD 1C 82 6D
 - 5 letters changed: D5 5F 20 9A C3 AA DD FC C3 94 F0 73 C6 79 DC 27
 - 10 letters changed: F1 C9 AE 2E F9 A2 DB 19 DF OC 50 B3 94 A2 18 86
- **SHA-1:**
 - Original: 5B OE D5 F4 99 F9 70 40 92 C4 95 16 C6 24 56 70 D4 C8 35 82
 - 1 letter changed: 32 06 E6 90 8F 75 14 BF 15 5B 6B DB 55 03 68 0C 06 48 35 3C
 - 5 letters changed: 31 8F 59 DA 04 00 BF 9D E7 75 4F 49 B9 3D 46 11 B4 A6 66 68
- **SHA-512:**
 - Modifications of 1, 5, and 10 letters produced entirely distinct hash strings with no visual similarities.
- **Conclusions:** Changing just one letter in the source text completely alters the resulting hash, with about half of the characters changing completely. This is known as the avalanche effect, ensuring the original text cannot be guessed simply by looking at the hash.

Task 3.2: Hash Collision Generation

- **MD5:**
 - 32-bit collision: Successful.
 - 64-bit collision: ~12 hours remaining.
 - 128-bit collision: ~7.9e+092 years remaining.
- **SHA-1:**
 - 32-bit collision: Successful.
 - 64-bit collision: ~11 hours remaining.
 - 128-bit collision: ~4.7e+092 years remaining.
- **Conclusions:** Older hashing algorithms like MD5 and SHA-1 are vulnerable and can be broken, but generating collisions is only feasible in a reasonable timeframe for smaller bit sizes (like 32 and 64 bits).

Task 3.3: Examining Known Collisions

- **Results:** Both the harmless hello.exe and the malicious erase.exe programs produced the exact same MD5 hash: CD C4 7D 67 01 59 EE F6 09 16 CA 03 A9 D4 A0 07.
- **Conclusions:** Because two entirely different files can generate the exact same MD5 hash (a collision), MD5 cannot be trusted to verify if a file is safe or authentic.

Task 3.4: Scripting MD5 in MATLAB

- **Results:** The patched MATLAB script output the identical hash (cdc47d670159eef60916ca03a9d4a007) for the files as the CrypTool program did.
- **Conclusions:** Hashing is fundamentally a mathematical formula. Processing the same file through the algorithm will always yield the exact same result, regardless of the software tool being used.

Task 3.5: Rainbow Tables Attack

- **Results:** MD5 hashes were generated for three passwords: *pass123*, *hasl0*, and *abc123def*.

The password recovery phase was skipped because the rainbow file could not be opened in the application.

Task 3.6: Rainbow Table Resource Estimation

Entirely skipped due to technical issues.

Task 3.7: Brute Force Attack

- **Results:**
 - **6-Character Password (123abc):** Cracked successfully almost immediately (1 of 1 hashes cracked for both MD5 and SHA-1).
 - **8-Character Password:** Estimated remaining time for MD5 was ~2.4 days; for SHA-1 it was ~4.3 days.
 - **Unknown Length (1-16 Characters):** Estimated remaining time exploded to roughly 1.91596e+010 years.
- **Conclusions:** Brute force attacks (testing every possible combination) are extremely slow and can take hours, days, or even years depending on the password's length. By comparison, rainbow tables can find the same password in seconds, provided the password length is known.

Task 3.8: Hash Function Pros and Cons

- **Pros:**
 - **One-Way Operation:** They are mathematically irreversible; you cannot reconstruct the original file or text from the hash alone.
 - **High Speed:** Hash algorithms process data and generate outputs very quickly.
 - **Fixed Output Length:** Regardless of whether the input is a single character or a massive multi-gigabyte file, the resulting hash will always have the exact same length.
- **Cons:**
 - **Vulnerable Legacy Algorithms:** Older functions (like MD5 and SHA-1) are broken and susceptible to collision attacks, meaning they can be tricked by malicious actors.
 - **Susceptible to Guessing:** If extra random data (known as "salt") is not

added before hashing, simple or common passwords can be easily guessed using brute-force tools or precomputed rainbow tables.

- **Constant Need for Upgrades:** Because computers continuously get faster and more powerful over time, older hash systems become obsolete and must be frequently updated to maintain security.